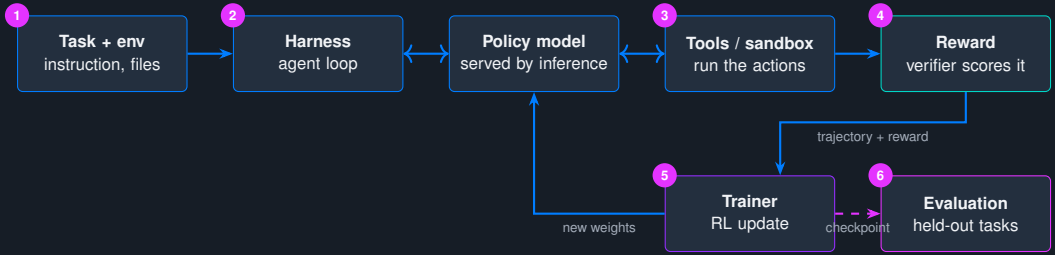
The agentic RL workflow

A model learns a multi-step job by trying it in a real environment many times and keeping what scores well.



1. **Pick a task** and start a fresh environment for it.
2. The **harness** runs the agent loop: ask the model, parse its tool call, return the result, repeat.
3. **Tools** act in an isolated **sandbox** (shell, files, APIs, a GPU), so mistakes stay contained.
4. When the agent stops, a **verifier** checks the end state and gives a reward (e.g. hidden tests pass).
5. The **trainer** compares several attempts at the same task and makes the better ones more likely (e.g. GRPO).
6. **Evaluation** on held-out tasks shows whether the gain is real, not memorized.

The components, and what each one does

Each row is something you own or we build together. Examples are from our Terminal-Bench + KernelBench demo on AgentCore.

Component	Role	Example
Policy model	the model being trained; it chooses every action	Qwen3-4B / Qwen3.5-9B, open weights
Tasks + training data	what the agent practises on, with a train/eval split	$\sim$200 Terminal-Bench shell tasks and 100 KernelBench GPU problems, plus held-out eval sets
Task definition + reward	what "done" means, scored automatically for each task type	hidden pytest oracle (1 / partial / 0); KernelBench: correct = 0.5, correct and $\geq 1.1\times$ faster = 1
Harness	the agent loop: prompt, tool-call parsing, turn and token budgets	a mini Claude Code loop (8 turns); the real Claude Code CLI as an option
Tools + skills	the actions the agent may take, and how-to guides it can read on demand	`bash`, `write_file`, `read_file`; skills such as "self-check your kernel"
Sandbox environment	an isolated, fresh machine per attempt, where the tools run	AgentCore microVM; EC2 for multi-container tasks; g6 GPU instance
Trainer + inference	generate attempts fast, then update the weights	slime (Megatron + SGLang), fully-async GRPO on $8\times$L40S
Evaluation	measures the gain on tasks the model never trained on	held-out solve rate every 10 steps, plus cost and latency per task

Questions to answer before our deep-dive

Area	Key questions	Why we ask
1 Goal and success	What should the agent do better than today? Which KPI does it move, and what number counts as success? What fails with today's approach (prompting, SFT, an API model)?	RL optimizes exactly what we measure, so we must agree on it first.
2 Model	Base model and size? Open weights or API-only? Licence limits on fine-tuning? Starting checkpoint? Serving target (latency, cost, hardware)?	RL needs the weights; model size sets GPU count and step time.
3 Tasks and data	Which task types? How many tasks of each type, and where do they come from (logs, a benchmark, synthetic)? Train/eval split? PII or contamination risk? Past trajectories?	Too few or too-easy tasks give no learning signal; the split decides what eval proves.
4 Task + reward per type	For each task type: input, expected output, verifier (tests, exact match, a checker, an LLM judge), reward (binary or partial)? Does it run automatically, in seconds? How could it be gamed?	The reward is the whole training signal. If it can be gamed, it will be.
5 Harness, tools, skills	Which harness (Claude Code, Codex, LangGraph, your own)? Tool APIs with their schemas: read-only or side-effecting, rate limits? Existing skills or playbooks? Turn and token budget?	Train in the same loop the model will be deployed in, or the gains don't transfer.
6 Sandbox environment	What does one episode need: OS/arch, image size, CPU or GPU, network, multi-container, secrets? Episode length? How many in parallel? Must state reset between episodes?	This decides the sandbox type (microVM, EC2, GPU) and the cost per episode.
7 Evaluation	Metrics (solve rate, pass@k, cost and latency per task, safety)? Held-out set or public benchmark? Baseline to beat? Human or LLM-judge spot checks?	Without a fixed eval and a baseline, a better training curve proves nothing.
8 Constraints	Compute budget, AWS region/account? Data residency, security and legal reviews? Timeline? Owners for data, environment, reward and eval?	This sets the scope, and who unblocks each piece.

Please bring: 5 example tasks, each with its expected output and how you grade it today.